Deducción natural en lógica de primer orden

Tema 2 — Razonamiento Automático

Fabián Fernando Serrano Suárez

Isabelle/HOL

Ciencias de la Computación

Recorrido

Qué cambia en primer orden

Recorrido

Qué cambia en primer orden

Cuantificador universal

Recorrido

Qué cambia en primer orden

Cuantificador universal

Cuantificador existencial

Recorrido

Qué cambia en primer orden

Cuantificador universal

Cuantificador existencial

Demostración de equivalencias

Recorrido

Qué cambia en primer orden

Cuantificador universal

Cuantificador existencial

Demostración de equivalencias

Igualdad

Recorrido

Qué cambia en primer orden

Cuantificador universal

Cuantificador existencial

Demostración de equivalencias

Igualdad

Resumen

Qué cambia en primer orden

De proposiciones a predicados

- En el Tema 1 las fórmulas eran átomos: p, q.

De proposiciones a predicados

- En el Tema 1 las fórmulas eran átomos: `p`, `q`.
- Ahora hablamos de objetos: `P(c)` dice que el objeto `c` tiene la propiedad `P`.

De proposiciones a predicados

- En el Tema 1 las fórmulas eran átomos: `p`, `q`.
- Ahora hablamos de objetos: `P(c)` dice que el objeto `c` tiene la propiedad `P`.
- Y aparecen los cuantificadores: $\forall x.\ P\,x$ «todo x cumple P» y $\exists x.\ P\,x$ «algún x cumple P».

De proposiciones a predicados

- En el Tema 1 las fórmulas eran átomos: `p`, `q`.
- Ahora hablamos de objetos: `P(c)` dice que el objeto `c` tiene la propiedad `P`.
- Y aparecen los cuantificadores: $\forall x.\ P\,x$ «todo x cumple P» y $\exists x.\ P\,x$ «algún x cumple P».

Todas las reglas del Tema 1 siguen valiendo.
Se añaden cuatro, una pareja por cuantificador, y dos para la igualdad.

Dos comandos nuevos de Isar

`fix a`

«Sea a un objeto cualquiera».

Se usa para probar un $\forall$: si se demuestra
`P a` sin saber nada de a, vale para todos.

Dos comandos nuevos de Isar

`fix a`

«Sea a un objeto cualquiera».

Se usa para probar un $\forall$: si se demuestra `P a` sin saber nada de a, vale para todos.

`obtain a where "P a"`

«Sea a un objeto que cumple `P`».

Se usa para aprovechar un $\exists$: de «algún x cumple P» se toma un testigo con nombre.

Dos comandos nuevos de Isar

`fix a`

«Sea a un objeto cualquiera».

Se usa para probar un $\forall$: si se demuestra `P a` sin saber nada de a, vale para todos.

`obtain a where "P a"`

«Sea a un objeto que cumple `P`».

Se usa para aprovechar un $\exists$: de «algún x cumple P» se toma un testigo con nombre.

La diferencia es de fondo: de `a` fijado con `fix` no se sabe nada; de `a` obtenido con `obtain` se sabe exactamente `P a`.

Otra vez dos niveles

Nivel	Implicación	Para todo

Otra vez dos niveles

Nivel	Implicación	Para todo
fórmula (objeto)	$\longrightarrow$	$\forall$

Otra vez dos niveles

Nivel	Implicación	Para todo
fórmula (objeto)	$\longrightarrow$	$\forall$
regla (meta)	$\Longrightarrow$	$\bigwedge$

Otra vez dos niveles

Nivel	Implicación	Para todo
fórmula (objeto)	$\longrightarrow$	$\forall$
regla (meta)	$\Longrightarrow$	$\bigwedge$

Es la misma distinción del Tema 1, ahora también para el cuantificador.

$\forall x.\ P\,x$ es una fórmula; $\bigwedge x.\ P\,x$ dice «para un x arbitrario se puede deducir $P\,x$».

Cuantificador universal

Las reglas del cuantificador universal

Reglas de deducción usadas

```
allE: ⟦∀x. P x; P a ⟹ R⟧ ⟹ R
allI: (⋀x. P x) ⟹ ∀x. P x
```

Las reglas del cuantificador universal

Reglas de deducción usadas

```
allE: ⟦∀x. P x; P a ⟹ R⟧ ⟹ R
allI: (⋀x. P x) ⟹ ∀x. P x
```

- **allE** *elimina*: de «todos cumplen P» se deduce P para cualquier objeto concreto a.

Las reglas del cuantificador universal

Reglas de deducción usadas

```
allE: ⟦∀x. P x; P a ⟹ R⟧ ⟹ R
allI: (⋀x. P x) ⟹ ∀x. P x
```

- **allE** *elimina*: de «todos cumplen P» se deduce P para cualquier objeto concreto a.
- **allI** *introduce*: si $P\,x$ se deduce para un x arbitrario, entonces vale $\forall x.\ P\,x$.

Las reglas del cuantificador universal

Reglas de deducción usadas

```
allE: ⟦∀x. P x; P a ⟹ R⟧ ⟹ R
allI: (⋀x. P x) ⟹ ∀x. P x
```

- **allE** *elimina*: de «todos cumplen P» se deduce P para cualquier objeto concreto a.
- **allI** *introduce*: si $P\,x$ se deduce para un x arbitrario, entonces vale $\forall x.\ P\,x$.

Eliminar es fácil: se elige el objeto. Introducir exige que el objeto no tenga nada de especial, y eso es lo que garantiza `fix`.

Ejemplo 1: P(c), ∀x. (P(x) ⟶ ¬Q(x)) ⊢ ¬Q(c)

```
lemma ejemplo_1a:
  assumes 1: "P(c)" and
          2: "∀x. (P(x) ⟶ ¬Q(x))"
  shows "¬Q(c)"
proof -
  have 3: "P(c) ⟶ ¬Q(c)"
    using 2 by (rule allE)
  show 4: "¬Q(c)"
    using 3 1 by (rule mp)
qed
```

Ejemplo 1: P(c), ∀x. (P(x) ⟶ ¬Q(x)) ⊢ ¬Q(c)

```
lemma ejemplo_1a:
  assumes 1: "P(c)" and
          2: "∀x. (P(x) ⟶ ¬Q(x))"
  shows "¬Q(c)"
proof -
  have 3: "P(c) ⟶ ¬Q(c)"
    using 2 by (rule allE)
  show 4: "¬Q(c)"
    using 3 1 by (rule mp)
qed
```

Reglas de deducción usadas

```
allE: ⟦∀x. P x; P a ⟹ R⟧ ⟹ R
mp: ⟦P ⟶ Q; P⟧ ⟹ Q
```

Ejemplo 1: P(c), ∀x. (P(x) ⟶ ¬Q(x)) ⊢ ¬Q(c)

```
lemma ejemplo_1a:
  assumes 1: "P(c)" and
          2: "∀x. (P(x) ⟶ ¬Q(x))"
  shows "¬Q(c)"
proof -
  have 3: "P(c) ⟶ ¬Q(c)"
    using 2 by (rule allE)
  show 4: "¬Q(c)"
    using 3 1 by (rule mp)
qed
```

Reglas de deducción usadas

```
allE: ⟦∀x. P x; P a ⟹ R⟧ ⟹ R
mp: ⟦P ⟶ Q; P⟧ ⟹ Q
```

- **Paso 3:** de la hipótesis universal se toma el caso particular `x = c`.

Ejemplo 1: P(c), ∀x. (P(x) ⟶ ¬Q(x)) ⊢ ¬Q(c)

```
lemma ejemplo_1a:
  assumes 1: "P(c)" and
          2: "∀x. (P(x) ⟶ ¬Q(x))"
  shows "¬Q(c)"
proof -
  have 3: "P(c) ⟶ ¬Q(c)"
    using 2 by (rule allE)
  show 4: "¬Q(c)"
    using 3 1 by (rule mp)
qed
```

Reglas de deducción usadas

```
allE: ⟦∀x. P x; P a ⟹ R⟧ ⟹ R
mp: ⟦P ⟶ Q; P⟧ ⟹ Q
```

- **Paso 3:** de la hipótesis universal se toma el caso particular `x = c`.
- **Paso 4:** con `P(c)` y el modus ponens del Tema 1 se llega a `¬Q(c)`.

Ejemplo 2: ∀x. (P x ⟶ ¬(Q x)), ∀x. P x ⊢ ∀x. ¬(Q x)

```
lemma ejemplo_2a:
  assumes 1: "∀x. (P x ⟶ ¬(Q x))" and
          2: "∀x. P x"
  shows "∀x. ¬(Q x)"
proof -
  { fix a
    have 3: "P a ⟶ ¬(Q a)"
      using 1 by (rule allE)
    have 4: "P a" using 2 by (rule allE)
    have 5: "¬(Q a)" using 3 4 by (rule mp) }
  thus "∀x. ¬(Q x)" by (rule allI)
qed
```

Ejemplo 2: ∀x. (P x ⟶ ¬(Q x)), ∀x. P x ⊢ ∀x. ¬(Q x)

```
lemma ejemplo_2a:
  assumes 1: "∀x. (P x ⟶ ¬(Q x))" and
          2: "∀x. P x"
  shows "∀x. ¬(Q x)"
proof -
  { fix a
    have 3: "P a ⟶ ¬(Q a)"
      using 1 by (rule allE)
    have 4: "P a" using 2 by (rule allE)
    have 5: "¬(Q a)" using 3 4 by (rule mp) }
  thus "∀x. ¬(Q x)" by (rule allI)
qed
```

- `fix a` abre un bloque con un objeto arbitrario.

Ejemplo 2: ∀x. (P x ⟶ ¬(Q x)), ∀x. P x ⊢ ∀x. ¬(Q x)

```
lemma ejemplo_2a:
  assumes 1: "∀x. (P x ⟶ ¬(Q x))" and
          2: "∀x. P x"
  shows "∀x. ¬(Q x)"
proof -
  { fix a
    have 3: "P a ⟶ ¬(Q a)"
      using 1 by (rule allE)
    have 4: "P a" using 2 by (rule allE)
    have 5: "¬(Q a)" using 3 4 by (rule mp) }
  thus "∀x. ¬(Q x)" by (rule allI)
qed
```

- `fix a` abre un bloque con un objeto arbitrario.
- Dentro se razona sobre a con **allE** y **mp**, sin suponer nada de él.

Ejemplo 2: ∀x. (P x ⟶ ¬(Q x)), ∀x. P x ⊢ ∀x. ¬(Q x)

```
lemma ejemplo_2a:
  assumes 1: "∀x. (P x ⟶ ¬(Q x))" and
          2: "∀x. P x"
  shows "∀x. ¬(Q x)"
proof -
  { fix a
    have 3: "P a ⟶ ¬(Q a)"
      using 1 by (rule allE)
    have 4: "P a" using 2 by (rule allE)
    have 5: "¬(Q a)" using 3 4 by (rule mp) }
  thus "∀x. ¬(Q x)" by (rule allI)
qed
```

- `fix a` abre un bloque con un objeto arbitrario.
- Dentro se razona sobre a con **allE** y **mp**, sin suponer nada de él.
- Al cerrar el bloque, **allI** convierte «para un a arbitrario, `¬(Q a)`» en `∀x. ¬(Q x)`.

Ejemplo 2: ∀x. (P x ⟶ ¬(Q x)), ∀x. P x ⊢ ∀x. ¬(Q x)

```
lemma ejemplo_2a:
  assumes 1: "∀x. (P x ⟶ ¬(Q x))" and
          2: "∀x. P x"
  shows "∀x. ¬(Q x)"
proof -
  { fix a
    have 3: "P a ⟶ ¬(Q a)"
      using 1 by (rule allE)
    have 4: "P a" using 2 by (rule allE)
    have 5: "¬(Q a)" using 3 4 by (rule mp) }
  thus "∀x. ¬(Q x)" by (rule allI)
qed
```

- `fix a` abre un bloque con un objeto arbitrario.
- Dentro se razona sobre a con **allE** y **mp**, sin suponer nada de él.
- Al cerrar el bloque, **allI** convierte «para un a arbitrario, `¬(Q a)`» en `∀x. ¬(Q x)`.

*Es el mismo patrón que **impI** con bloques en el Tema 1: el bloque se convierte en la fórmula.*

Cuantificador existencial

Las reglas del cuantificador existencial

Reglas de deducción usadas

```
exI: P a ⟹ ∃x. P x
exE: ⟦∃x. P x; ⋀x. P x ⟹ Q⟧ ⟹ Q
```

Las reglas del cuantificador existencial

Reglas de deducción usadas

```
exI: P a ⟹ ∃x. P x
exE: ⟦∃x. P x; ⋀x. P x ⟹ Q⟧ ⟹ Q
```

- **exI** *introduce*: basta exhibir un objeto a que cumpla P.

Las reglas del cuantificador existencial

Reglas de deducción usadas

```
exI: P a ⟹ ∃x. P x
exE: ⟦∃x. P x; ⋀x. P x ⟹ Q⟧ ⟹ Q
```

- **exI** *introduce*: basta exhibir un objeto a que cumpla P.
- **exE** *elimina*: de «alguno cumple P» se puede concluir Q si Q se deduce para un testigo arbitrario.

Las reglas del cuantificador existencial

Reglas de deducción usadas

```
exI: P a ⟹ ∃x. P x
exE: ⟦∃x. P x; ⋀x. P x ⟹ Q⟧ ⟹ Q
```

- **exI** *introduce*: basta exhibir un objeto a que cumpla P.
- **exE** *elimina*: de «alguno cumple P» se puede concluir Q si Q se deduce para un testigo arbitrario.

En Isar, ***exE*** *se usa casi siempre a través de* `obtain ... where ... by (rule exE)`.

Ejemplo 3: ∀x. P x ⊢ ∃x. P x

```
lemma ejemplo_3a:
  assumes "∀x. P x"
  shows "∃x. P x"
proof -
  fix a
  have "P a" using assms by (rule allE)
  thus "∃x. P x" by (rule exI)
qed
```

Ejemplo 3: ∀x. P x ⊢ ∃x. P x

```
lemma ejemplo_3a:
  assumes "∀x. P x"
  shows "∃x. P x"
proof -
  fix a
  have "P a" using assms by (rule allE)
  thus "∃x. P x" by (rule exI)
qed
```

- Se toma un objeto cualquiera a, se particulariza el ∀ en él, y ese mismo a sirve de testigo del ∃.

Ejemplo 3: ∀x. P x ⊢ ∃x. P x

```
lemma ejemplo_3a:
  assumes "∀x. P x"
  shows "∃x. P x"
proof -
  fix a
  have "P a" using assms by (rule allE)
  thus "∃x. P x" by (rule exI)
qed
```

- Se toma un objeto cualquiera a, se particulariza el ∀ en él, y ese mismo a sirve de testigo del ∃.

¿Por qué es válido? *En HOL todo tipo tiene al menos un elemento; con dominios vacíos no lo sería.*

Ejemplo 4: ∀x. (P x ⟶ Q x), ∃x. P x ⊢ ∃x. Q x

```
lemma ejemplo_4a:
  assumes 1: "∀x. (P x ⟶ Q x)" and
          2: "∃x. P x"
  shows "∃x. Q x"
proof -
  obtain a where 3: "P a"
    using 2 by (rule exE)
  have 4: "P a ⟶ Q a" using 1 by (rule allE)
  have 5: "Q a" using 4 3 by (rule mp)
  thus 6: "∃x. Q x" by (rule exI)
qed
```

Ejemplo 4: ∀x. (P x ⟶ Q x), ∃x. P x ⊢ ∃x. Q x

```
lemma ejemplo_4a:
  assumes 1: "∀x. (P x ⟶ Q x)" and
          2: "∃x. P x"
  shows "∃x. Q x"
proof -
  obtain a where 3: "P a"
    using 2 by (rule exE)
  have 4: "P a ⟶ Q a" using 1 by (rule allE)
  have 5: "Q a" using 4 3 by (rule mp)
  thus 6: "∃x. Q x" by (rule exI)
qed
```

- **Paso 3:** `obtain` toma un testigo `a` del ∃: ahora sabemos `P a`.

Ejemplo 4: $\forall x.\ (P\ x \longrightarrow Q\ x),\ \exists x.\ P\ x \vdash \exists x.\ Q\ x$

```
lemma ejemplo_4a:
  assumes 1: "∀x. (P x ⟶ Q x)" and
          2: "∃x. P x"
  shows "∃x. Q x"
proof -
  obtain a where 3: "P a"
    using 2 by (rule exE)
  have 4: "P a ⟶ Q a" using 1 by (rule allE)
  have 5: "Q a" using 4 3 by (rule mp)
  thus 6: "∃x. Q x" by (rule exI)
qed
```

- **Paso 3:** `obtain` toma un testigo `a` del $\exists$: ahora sabemos `P a`.
- **Pasos 4 y 5:** se particulariza el $\forall$ en ese mismo `a` y se aplica modus ponens.

Ejemplo 4: $\forall x.\ (P\ x \longrightarrow Q\ x),\ \exists x.\ P\ x \vdash \exists x.\ Q\ x$

```
lemma ejemplo_4a:
  assumes 1: "∀x. (P x ⟶ Q x)" and
          2: "∃x. P x"
  shows "∃x. Q x"
proof -
  obtain a where 3: "P a"
    using 2 by (rule exE)
  have 4: "P a ⟶ Q a" using 1 by (rule allE)
  have 5: "Q a" using 4 3 by (rule mp)
  thus 6: "∃x. Q x" by (rule exI)
qed
```

- **Paso 3:** `obtain` toma un testigo a del $\exists$: ahora sabemos `P a`.
- **Pasos 4 y 5:** se particulariza el $\forall$ en ese mismo a y se aplica modus ponens.
- **Paso 6:** a es testigo de $\exists x.\ Q\ x$.

Ejemplo 4: $\forall x.\ (P\ x \longrightarrow Q\ x),\ \exists x.\ P\ x \vdash \exists x.\ Q\ x$

```
lemma ejemplo_4a:
  assumes 1: "∀x. (P x ⟶ Q x)" and
          2: "∃x. P x"
  shows "∃x. Q x"
proof -
  obtain a where 3: "P a"
    using 2 by (rule exE)
  have 4: "P a ⟶ Q a" using 1 by (rule allE)
  have 5: "Q a" using 4 3 by (rule mp)
  thus 6: "∃x. Q x" by (rule exI)
qed
```

- **Paso 3:** `obtain` toma un testigo a del $\exists$: ahora sabemos `P a`.
- **Pasos 4 y 5:** se particulariza el $\forall$ en ese mismo a y se aplica modus ponens.
- **Paso 6:** a es testigo de $\exists x.\ Q\ x$.

*Las cuatro reglas de los cuantificadores, más **mp**, en una sola prueba.*

Demostración de equivalencias

Ejemplo 6.1: $\forall x.\ P(x) \wedge Q(x) \vdash (\forall x.\ P(x)) \wedge (\forall x.\ Q(x))$

```
lemma ejemplo_6_1a:
  assumes "∀x. P(x) ∧ Q(x)"
  shows "(∀x. P(x)) ∧ (∀x. Q(x))"
proof (rule conjI)
  show "∀x. P(x)"
  proof (rule allI)
     fix a
     have "P(a) ∧ Q(a)" using assms by (rule allE)
     thus "P(a)" by (rule conjunct1)
  qed
next
  show "∀x. Q(x)"
  proof (rule allI)
     fix a
     have "P(a) ∧ Q(a)" using assms by (rule allE)
     thus "Q(a)" by (rule conjunct2)
  qed
qed
```

Ejemplo 6.1: ∀x. P(x) ∧ Q(x) ⊢ (∀x. P(x)) ∧ (∀x. Q(x))

```
lemma ejemplo_6_1a:
  assumes "∀x. P(x) ∧ Q(x)"
  shows "(∀x. P(x)) ∧ (∀x. Q(x))"
proof (rule conjI)
  show "∀x. P(x)"
  proof (rule allI)
     fix a
     have "P(a) ∧ Q(a)" using assms by (rule allE)
     thus "P(a)" by (rule conjunct1)
  qed
next
  show "∀x. Q(x)"
  proof (rule allI)
     fix a
     have "P(a) ∧ Q(a)" using assms by (rule allE)
     thus "Q(a)" by (rule conjunct2)
  qed
qed
```

- **Hacia atrás:** la conclusión es una conjunción, así que se empieza por **conjI**. Quedan dos objetivos, separados por `next`.

Ejemplo 6.1: $\forall x.\ P(x) \wedge Q(x) \vdash (\forall x.\ P(x)) \wedge (\forall x.\ Q(x))$

```
lemma ejemplo_6_1a:
  assumes "∀x. P(x) ∧ Q(x)"
  shows "(∀x. P(x)) ∧ (∀x. Q(x))"
proof (rule conjI)
  show "∀x. P(x)"
  proof (rule allI)
     fix a
     have "P(a) ∧ Q(a)" using assms by (rule allE)
     thus "P(a)" by (rule conjunct1)
  qed
next
  show "∀x. Q(x)"
  proof (rule allI)
     fix a
     have "P(a) ∧ Q(a)" using assms by (rule allE)
     thus "Q(a)" by (rule conjunct2)
  qed
qed
```

- **Hacia atrás:** la conclusión es una conjunción, así que se empieza por **conjI**. Quedan dos objetivos, separados por `next`.
- Cada objetivo es un $\forall$: se abre con **allI** y `fix a`.

Ejemplo 6.1: $\forall x.\ P(x) \wedge Q(x) \vdash (\forall x.\ P(x)) \wedge (\forall x.\ Q(x))$

```
lemma ejemplo_6_1a:
  assumes "∀x. P(x) ∧ Q(x)"
  shows "(∀x. P(x)) ∧ (∀x. Q(x))"
proof (rule conjI)
  show "∀x. P(x)"
  proof (rule allI)
     fix a
     have "P(a) ∧ Q(a)" using assms by (rule allE)
     thus "P(a)" by (rule conjunct1)
  qed
next
  show "∀x. Q(x)"
  proof (rule allI)
     fix a
     have "P(a) ∧ Q(a)" using assms by (rule allE)
     thus "Q(a)" by (rule conjunct2)
  qed
qed
```

- **Hacia atrás:** la conclusión es una conjunción, así que se empieza por **conjI**. Quedan dos objetivos, separados por `next`.
- Cada objetivo es un $\forall$: se abre con **allI** y `fix a`.
- Dentro, **allE** y **conjunct1** o **conjunct2**.

Ejemplo 6.1: ∀x. P(x) ∧ Q(x) ⊢ (∀x. P(x)) ∧ (∀x. Q(x))

```
lemma ejemplo_6_1a:
  assumes "∀x. P(x) ∧ Q(x)"
  shows "(∀x. P(x)) ∧ (∀x. Q(x))"
proof (rule conjI)
  show "∀x. P(x)"
  proof (rule allI)
     fix a
     have "P(a) ∧ Q(a)" using assms by (rule allE)
     thus "P(a)" by (rule conjunct1)
  qed
next
  show "∀x. Q(x)"
  proof (rule allI)
     fix a
     have "P(a) ∧ Q(a)" using assms by (rule allE)
     thus "Q(a)" by (rule conjunct2)
  qed
qed
```

- **Hacia atrás:** la conclusión es una conjunción, así que se empieza por **conjI**. Quedan dos objetivos, separados por `next`.
- Cada objetivo es un ∀: se abre con **allI** y `fix a`.
- Dentro, **allE** y **conjunct1** o **conjunct2**.

La forma de la conclusión dicta la estructura de la prueba: primero ∧, después ∀.

Ejemplo 5.1: ¬∀x. P x ⊢ ∃x. ¬(P x)

```
lemma ejemplo_5_1a:
  assumes "¬(∀x. P(x))"
  shows "∃x. ¬P(x)"
proof (rule ccontr)
  assume "¬(∃x. ¬P(x))"
  have "∀x. P(x)"
  proof (rule allI)
    fix a
    show "P(a)"
    proof (rule ccontr)
      assume "¬P(a)"
      hence "∃x. ¬P(x)" by (rule exI)
      with ‘¬(∃x. ¬P(x))‘ show False by (rule notE)
    qed
  qed
  with assms show False by (rule notE)
qed
```

Ejemplo 5.1: ¬∀x. P x ⊢ ∃x. ¬(P x)

```
lemma ejemplo_5_1a:
  assumes "¬(∀x. P(x))"
  shows "∃x. ¬P(x)"
proof (rule ccontr)
  assume "¬(∃x. ¬P(x))"
  have "∀x. P(x)"
  proof (rule allI)
    fix a
    show "P(a)"
    proof (rule ccontr)
      assume "¬P(a)"
      hence "∃x. ¬P(x)" by (rule exI)
      with ‘¬(∃x. ¬P(x))‘ show False by (rule notE)
    qed
  qed
  with assms show False by (rule notE)
qed
```

- Se razona por contradicción dos veces, con **ccontr**: una para el ∃, otra para P(a).

Ejemplo 5.1: ¬∀x. P x ⊢ ∃x. ¬(P x)

```
lemma ejemplo_5_1a:
  assumes "¬(∀x. P(x))"
  shows "∃x. ¬P(x)"
proof (rule ccontr)
  assume "¬(∃x. ¬P(x))"
  have "∀x. P(x)"
  proof (rule allI)
    fix a
    show "P(a)"
    proof (rule ccontr)
      assume "¬P(a)"
      hence "∃x. ¬P(x)" by (rule exI)
      with ‘¬(∃x. ¬P(x))‘ show False by (rule notE)
    qed
  qed
  with assms show False by (rule notE)
qed
```

- Se razona por contradicción dos veces, con **ccontr**: una para el ∃, otra para P(a).
- Si ningún objeto cumpliera $\neg P$, todos cumplirían P, contra la hipótesis.

Ejemplo 5.1: ¬∀x. P x ⊢ ∃x. ¬(P x)

```
lemma ejemplo_5_1a:
  assumes "¬(∀x. P(x))"
  shows "∃x. ¬P(x)"
proof (rule ccontr)
  assume "¬(∃x. ¬P(x))"
  have "∀x. P(x)"
  proof (rule allI)
    fix a
    show "P(a)"
    proof (rule ccontr)
      assume "¬P(a)"
      hence "∃x. ¬P(x)" by (rule exI)
      with ‘¬(∃x. ¬P(x))‘ show False by (rule notE)
    qed
  qed
  with assms show False by (rule notE)
qed
```

- Se razona por contradicción dos veces, con **ccontr**: una para el ∃, otra para `P(a)`.
- Si ningún objeto cumpliera $\neg P$, todos cumplirían P, contra la hipótesis.
- Es una regla clásica: de «no todos» no se obtiene un testigo concreto, se llega por reducción al absurdo.

Igualdad

Las reglas de la igualdad

Reglas de deducción usadas

refl: $t = t$

subst: $[\![s = t;\ P\ s]\!] \Longrightarrow P\ t$

Las reglas de la igualdad

Reglas de deducción usadas

```
refl:  t = t
subst: ⟦s = t; P s⟧ ⟹ P t
```

- **refl**: todo objeto es igual a sí mismo.

Las reglas de la igualdad

Reglas de deducción usadas

```
refl:  t = t
subst: ⟦s = t; P s⟧ ⟹ P t
```

- **refl**: todo objeto es igual a sí mismo.
- **subst**: si `s = t`, en cualquier propiedad se puede sustituir `s` por `t`.

Las reglas de la igualdad

Reglas de deducción usadas

```
refl:  t = t
subst: ⟦s = t; P s⟧ ⟹ P t
```

- **refl**: todo objeto es igual a sí mismo.
- **subst**: si `s = t`, en cualquier propiedad se puede sustituir `s` por `t`.

Con solo estas dos se derivan la simetría y la transitividad, como muestran los ejemplos siguientes.

Ejemplos 10 y 11: transitividad y simetría

$$x = y,\ y = z \vdash x = z$$

```
lemma ejemplo_10a:
  assumes "x = y"
          "y = z"
  shows "x = z"
proof -
  show "x = z"
    using assms(2,1) by (rule subst)
qed
```

Ejemplos 10 y 11: transitividad y simetría

$x = y,\ y = z \vdash x = z$

```
lemma ejemplo_10a:
  assumes "x = y"
          "y = z"
  shows "x = z"
proof -
  show "x = z"
    using assms(2,1) by (rule subst)
qed
```

En `x = y` se sustituye `y` por `z`, usando `y = z`.

Ejemplos 10 y 11: transitividad y simetría

`x = y, y = z ⊢ x = z`

```
lemma ejemplo_10a:
  assumes "x = y"
          "y = z"
  shows "x = z"
proof -
  show "x = z"
    using assms(2,1) by (rule subst)
qed
```

En `x = y` se sustituye `y` por `z`, usando `y = z`.

`s = t ⊢ t = s`

```
lemma ejemplo_11a:
  assumes "s = t"
  shows "t = s"
proof -
  have "s = s" by (rule refl)
  with assms show "t = s"
    by (rule subst)
qed
```

Ejemplos 10 y 11: transitividad y simetría

`x = y, y = z ⊢ x = z`

```
lemma ejemplo_10a:
  assumes "x = y"
          "y = z"
  shows "x = z"
proof -
  show "x = z"
    using assms(2,1) by (rule subst)
qed
```

En `x = y` se sustituye `y` por `z`, usando `y = z`.

`s = t ⊢ t = s`

```
lemma ejemplo_11a:
  assumes "s = t"
  shows "t = s"
proof -
  have "s = s" by (rule refl)
  with assms show "t = s"
    by (rule subst)
qed
```

En `s = s` se sustituye la primera `s` por `t`.

Resumen

Las reglas del tema

Introducción	Eliminación

Las reglas del tema

	Introducción	Eliminación
$\forall$	**allI**	**allE**

Las reglas del tema

	Introducción	Eliminación
$\forall$	**allI**	**allE**
$\exists$	**exI**	**exE** (con `obtain`)

Las reglas del tema

	Introducción	Eliminación
$\forall$	**allI**	**allE**
$\exists$	**exI**	**exE** (con `obtain`)
$=$	**refl**	**subst**

Las reglas del tema

	Introducción	Eliminación
$\forall$	**allI**	**allE**
$\exists$	**exI**	**exE** (con `obtain`)
$=$	**refl**	**subst**

Todas las reglas del Tema 1 siguen disponibles.

- `fix a`: un objeto cualquiera, para probar un $\forall$ con **allI**.

- `fix a`: un objeto cualquiera, para probar un $\forall$ con **allI**.
- `obtain a where`: un objeto concreto, para usar un $\exists$ con **exE**.

- `fix a`: un objeto cualquiera, para probar un $\forall$ con **allI**.
- `obtain a where`: un objeto concreto, para usar un $\exists$ con **exE**.
- $\forall$ es fórmula y $\bigwedge$ es regla, como $\longrightarrow$ y $\Longrightarrow$ en el Tema 1.

- `fix a`: un objeto cualquiera, para probar un $\forall$ con **allI**.
- `obtain a where`: un objeto concreto, para usar un $\exists$ con **exE**.
- $\forall$ es fórmula y $\bigwedge$ es regla, como $\longrightarrow$ y $\Longrightarrow$ en el Tema 1.
- La forma de la conclusión sugiere la estructura de la prueba: se empieza por su conectivo o cuantificador principal.